

Pipelines et opérations multicycles

Exercice 1

Un processeur P1 dispose du pipeline à 5 étages pour les opérations entières et les chargements/rangements flottants et des pipelines suivants pour les instructions flottantes

Multiplication : FMUL

LI	DI	LR	EX1	EX2	EX3	EX4	ER
----	----	----	-----	-----	-----	-----	----

Addition : FADD

LI	DI	LR	EX1	EX2	ER
----	----	----	-----	-----	----

Les instructions flottantes LF et ST ont les mêmes pipelines que les instructions entières LW et SW.

Donner les latences des instructions flottantes FMUL et FADD (une instruction i a une latence de n si l'instruction suivante peut commencer au cycle $i+n$; une latence de 1 signifie que l'instruction suivante peut commencer au cycle suivant).

Mêmes questions avec le processeur P2 suivant

Multiplication : FMUL

LI	DI	LR	EX1	EX2	EX3	EX4	EX5	EX6	ER
----	----	----	-----	-----	-----	-----	-----	-----	----

Addition : FADD

LI	DI	LR	EX1	EX2	EX3	EX4	ER
----	----	----	-----	-----	-----	-----	----

Exercice 2

On ajoute au jeu d'instructions NIOS II des instructions flottantes de simple précision (32 bits) (Figure 2) et 32 registres flottants F0 à F31 (F0 est un registre normal).

Les additions, soustractions et multiplications flottantes sont pipelinées. Une nouvelle instruction peut démarrer à chaque cycle. Les latences sont de 2 cycles pour LF et fournies par les résultats de l'exercice 1 pour les instructions flottantes de multiplication et d'addition.

La division a une latence de 30 cycles. Elle n'est pas pipelinée : une nouvelle division ne peut commencer que lorsque la division précédente est terminée.

Boucle SAXPY

Soit le programme suivant

```
float x[100], y[100], z[100] ;
```

```
main() {
    int i ;
    for (i=0 ; i<100 ; i++)
        z[i] = x[i]* y[i];
}
```

Quels sont les temps d'exécution par itération de la boucle avec le processeur P1 ? Avec le processeur P2 ?

On utilisera les hypothèses suivantes :

Les trois vecteurs X, Y et Z sont successivement rangés en mémoire.

R1 contient l'adresse de x[0].

R3 contient l'adresse de x[100], c'est-à-dire l'adresse du mot suivant le dernier élément du tableau x

Soient des déroulements de boucle d'ordre 2 et 4

```
int main() {
int i ;
for (i=0 ; i<100 ; i+=2) {
    z[i] = x[i]* y[i];
    z[i+1]= x[i+1]* y[i+1];}

main() {
int i ;
for (i=0 ; i<100 ; i+=4) {
    z[i] = x[i]* y[i];
    z[i+1]= x[i+1]* y[i+1];}
    z[i+2] = x[i+2]* y[i+2];
    z[i+3]= x[i+3]* y[i+3];}
```

Quels sont les temps d'exécution par itération de la boucle initiale pour les 4 cas

- déroulage d'ordre 2 avec P1
- déroulage d'ordre 4 avec P1
- déroulage d'ordre 2 avec P2
- déroulage d'ordre 4 avec P2

Quels sont les temps minimaux par itération de la boucle initiale que l'on peut obtenir par déroulage avec chaque processeur ?

Comment procède-t-on si le nombre d'itérations n'est pas un multiple du facteur de déroulage ?

Somme des carrés

Soit le programme suivant

```
float x[100], s ;

main() {
int i ;
for (i=1 ; i<100 ; i++)
    s+=x[i]*x[i];
}
```

Quels sont les temps d'exécution par itération de la boucle avec le processeur P1 ? Avec le processeur P2 ?

S est initialement dans le registre F0

R1 contient l'adresse de x[0].

R3 contient l'adresse de x[100], c'est-à-dire l'adresse du mot suivant le dernier élément du tableau x

Quels sont les temps d'exécution par itération de la boucle initiale pour les 4 cas

- déroulage d'ordre 2 avec P1
- déroulage d'ordre 4 avec P1
- déroulage d'ordre 2 avec P2

- déroulage d'ordre 4 avec P2

Quels sont les temps minimaux par itération de la boucle initiale que l'on peut obtenir par déroulage avec chaque processeur ?